

Sprint 14 · 资金路径 TOCTOU 竞态修复 & 并发安全压测报告

生成时间：2026-08-13 · 数据库：本地真实 MySQL · 环境：多进程真实行级并发（非 mock） 用途：周报同步 —— 原子性修复验收 + 并发压测日志摘要 + 关键指标对比

一、本轮结论（TL;DR）

维度	结果
TOCTOU 高风险点	✅ 5 处全部修复 (配额超发 / 支付重复开通扣分 / 模板重复购买分成 / 提现覆盖写 / 提现审核重复退款)
后端全量单测	✅ 639 / 639 通过 , 106 套件, 0 失败
S14 市场专项单测	✅ 29 / 29 通过 , 6 套件, 0 失败
审核状态机并发压测	✅ 全绿 (原子性 / 幂等 / 防重复 / 审计一致)
资金路径并发压测 (新增)	✅ 全绿, 3 轮 × 6 并发, 购买防重复扣分/分成、提现防覆盖写、审核防重复退款均成立
语法校验	✅ 5 个改动服务文件 <code>node --check</code> 全部通过

核心结论：所有资金/配额写路径已收敛为「原子条件 UPDATE + `.changes` 判定」或「用户锚点行 `FOR UPDATE` 串行化」范式，在多进程真实行锁竞争下未被击穿。

二、修复的 5 处高风险 TOCTOU 竞态

编号	位置	风险	修复手法
H1	<code>quotaService.tryConsumeGenerationBounded</code>	配额超发（多请求并发读到同一 <code>used</code> 值）	条件自增 <code>UPDATE ... SET used=used+1 WHERE used<limit</code> + 唯一键 INSERT 兜底
H2	<code>paymentService.handlePaymentSuccess</code>	支付回调重复开通会员 / 重复扣分	先原子抢占 <code>UPDATE ... SET pay_status='paid' WHERE pay_status='pending'</code> , <code>.changes</code> 判赢家
H3	<code>marketplaceService.acquireTemplate</code>	模板重复购买 / 重复扣分 / 重复分成	事务内锁用户行 + 复查已购 + <code>deductPointsAtomic</code> 原子扣分
H4	<code>creatorService.requestWithdrawal</code>	提现基于旧余额覆盖写 (lost update) 导致超额提现	原子冻结 <code>UPDATE ... SET balance=balance-amt WHERE balance>=amt</code> , <code>.changes=1</code> 才成立
H5	<code>creatorService.reviewWithdrawal</code>	审核并发导致重复退款 / 重复累加 <code>total_withdrawn</code>	原子状态流转 <code>UPDATE ... WHERE status IN ('pending','approved')</code> , 赢家唯一

三、资金路径并发压测（本轮重点）

脚本： `backend-node/scripts/stress_s14_finance_concurrency.js`

并发模型：后端 DB 层为 `sync-mysql` 单例同步连接，进程内 `Promise.all` 无法产生真正的 DB 并发，故采用**多进程**——父进程用真实服务层播种，派生 N 个 worker 子进程各持独立 `getDb()` 连接，在统一「起跑栅栏」时刻对同一目标发起资金操作，制造真实**行级争用**。

执行命令

```
cd backend-node
node scripts/stress_s14_finance_concurrency.js --workers 6 --rounds 3
```

参数：6 并发 · 3 轮 · 模板单价 3 元 (= 300 积分, 100 积分 = 1 元) · 平台分成费率 0.3

3.1 场景与断言覆盖

场景	竞态点	关键断言
多买家并发购买同一付费模板	H3	每人恰好 1 download + 1 settlement；各扣 300；创作者累计分成正确
同一买家并发购买同一模板	H3（幂等）	恰好 1 次真实购买，其余 alreadyOwned；仅扣 1 次 300
同一创作者并发多笔提现	H4	成功笔数 ≤ 理论上限；冻结额 == 成功额之和；余额永不为负
多管理员并发审核同一提现单	H5	恰好 1 个成功；退款流水唯一；余额只退回 1 次

3.2 压测日志摘要（3 轮全绿，exit 0）

Sprint 14 · 资金路径（购买/分成/提现）并发安全压测

并发 worker=6 · 轮次=3 · 单价=3元(=300积分)

— H3 多买家并发购买（每轮） —

✔ 每位买家各成功购买 1 次 — 成功=6/6

✔ download 记录数 == 买家数（无重复） — downloads=6

✔ settlement 记录数 == 买家数（无重复分成） — settlements=6

✔ 每位买家恰好扣 needPoints（无多扣/漏扣） — needPoints=300

✔ 创作者分成累计正确 — 实得=12.6 期望=12.6（费率=0.3）

— H3 同买家并发购买（幂等，每轮） —

✔ 恰好 1 次真实购买成功 — 成功=1

✔ 其余识别为 alreadyOwned（无异常） — alreadyOwned=5

✔ download 唯一 / settlement 唯一 / 仅扣 1 次 300

— H5 多管理员并发审核（每轮） —

并发数=6 · 成功=1 · NOT_REVIEWABLE=5 · 其他=0

✔ 恰好 1 个审核成功；终态=rejected

✔ 退款流水恰好 1 条（无重复退款）

✔ 余额只退回 1 次（== 冻结前）

— H4 同创作者并发提现（每轮） —

[第1轮] 余额=14.7 每笔=10 请求6笔 理论上限=1 → 成功=1 拒绝=5 (INSUFFICIENT_BALANCE)

[第3轮] 余额=24.1 每笔=10 请求6笔 理论上限=2 → 成功=2 拒绝=4 (INSUFFICIENT_BALANCE)

✔ 成功笔数不超过理论上限（无超额提现）

✔ 冻结金额 == 成功提现额之和（无 lost update）

✔ 余额永不为负 · 无异常失败码

压测结束，已清理压测模板 6 个 + 相关资金流水。

🔥 全部资金并发安全断言通过。

3.3 关键指标对比

指标	修复前（潜在风险行为）	修复后（实测结果）
多买家购买 · settlement 条数	可能 > 买家数（重复分成）	== 6（== 买家数）
同买家 6 并发购买 · 扣分次数	可能 6 次（重复扣 1800）	1 次（恰好 300）
同买家 6 并发购买 · download 条数	可能多条	1 条（唯一）
创作者分成累计（6 单 × 3 元 × 70%）	可能虚高	实得 12.6 == 期望 12.6
6 笔并发提现（余额 14.7，每笔 10）	可能成功 >1（超额提现）	成功 1，其余 5 笔 INSUFFICIENT_BALANCE
6 笔并发提现（余额 24.1，每笔 10）	可能覆盖写导致账目错乱	成功 2，冻结额 20 == 成功额之和
6 管理员并发驳回 · 退款流水	可能 6 条（重复退款）	1 条（唯一）
审核赢家	可能多个终态并存	恰好 1 个，其余 NOT_REVIEWABLE

亮点：H4 在不同余额下自动呈现「理论上限即成功上限」（14.7→1 笔、24.1→2 笔），证明原子条件扣减严格约束了并发提

现总额，杜绝了 lost update。

四、回归测试摘要

全量后端单测

```
cd backend-node && node --test test/*.test.js
# tests 639 · suites 106 · pass 639 · fail 0
```

S14 市场专项

```
cd backend-node && node --test test/s14TemplateMarketplace.test.js
# tests 29 · suites 6 · pass 29 · fail 0
```

审核状态机并发压测

```
node scripts/stress_s14_review_concurrency.js --workers 8 --rounds 3 --mixed
# 原子性 / 幂等 / 防重复 / 审计一致 全绿
```

五、环境清理与稳定性说明

项	状态
端口 5679（后端）	✅ 空闲，无残留 <code>node --watch</code> 实例
端口 3013 / 3014（前端）	🟢 两个 vite dev server 正常运行中（用户端 / 管理端）
残留 <code>md-to-pdf.mjs</code> 僵留进程	✅ 已清理（原运行 25–47 分钟的 2 个孤儿进程已终止）
孤儿 headless Chrome / puppeteer	✅ 无
压测残留数据	✅ 收尾自动清理，创作者聚合列复位归零， <code>point_logs</code> 无残留

说明：早前观察到的后端 `EADDRINUSE` 崩溃与 `sync-rpc` 超时，是 `node --watch` 热重启时端口占用 + 旧 `md-to-pdf` 进程僵留所致的环境冲突，非 TOCTOU 修复引入的代码缺陷。现已全部清理。

六、一键复现

```
# 资金路径并发压测
cd backend-node && node scripts/stress_s14_finance_concurrency.js --workers 6 --rounds 3

# 审核状态机并发压测
cd backend-node && node scripts/stress_s14_review_concurrency.js --workers 8 --rounds 3 --mixed

# 全量回归
cd backend-node && node --test test/*.test.js

# 生成本报告 PDF（可选）
npm run report:pdf docs/reports/Sprint14-资金路径TOCTOU并发压测报告.md docs/reports/Sprint14-资金路径TOCTO
```